Weakly-Supervised Progress Estimation for LLM Agent Trajectories

Anonymous Authors

Abstract

LLM-based agents frequently enter periods of *semantic stagnation*—executing superficially diverse but collectively unproductive action sequences during long-horizon tasks. Detecting such stagnation is difficult because individual actions may appear reasonable in isolation; the signal emerges only at the *window level*, across consecutive steps. We present a weakly-supervised framework for window-level progress estimation on agent trajectories. Three domain-specific labeling functions (command repetition, file-diff delta, outcome back-propagation) generate noisy training labels via majority vote, enabling scalable annotation without human effort. A lightweight GRU probe trained on these labels achieves 0.91 macro-F1 on high-confidence windows and remains competitive (0.82 F1) when trained on the full, noisier label set. On a 400-window human-annotated gold set, the probe reaches 0.77 macro-F1—outperforming the best heuristic baseline (Repeat-3, 0.72) and the labeling-function oracle (0.75), while costing 35× less per window than an LLM-as-judge (0.82 F1). Cross-agent transfer from SWE-agent to OpenHands yields 0.73 F1 zero-shot, recoverable to 0.82 with 200 labeled examples, demonstrating that the learned progress signal generalizes beyond a single agent implementation. Our primary contribution is the weak-supervision framework itself—a practical, extensible pipeline that makes progress estimation research feasible without expensive manual annotation.

1 Introduction

LLM-powered agents [**?**, **?**] increasingly tackle complex, multi-step software engineering tasks. A recurring failure mode is *semantic stagnation*: the agent issues a sequence of individually plausible actions that collectively make no progress toward the goal. Unlike syntactic loops, stagnation is semantically diverse—the agent may try different commands, edit different files, or consult different resources, yet remain stuck.

Existing monitoring approaches address related but distinct problems. Cycle detectors identify structural repetition but miss semantically diverse stagnation. Step-level value estimators score individual actions without modeling the temporal dynamics within a window. Self-reflection mechanisms rely on the stagnating agent itself to diagnose the problem.

We reframe progress monitoring as a **window-level binary classification** task: given a sliding window of k consecutive agent steps, predict whether the window exhibits *progress* or *stagnation*. This formulation captures the temporal signature of stagnation onset—a pattern that emerges across steps rather than within any single action.

The key bottleneck is *data*. Labeling thousands of trajectory windows for progress vs. stagnation requires expert annotators with task-level context—an expensive and non-scalable process. We address this with a **weak-supervision framework** built on the data programming paradigm. Three domain-specific labeling functions (LFs)—command repetition detection, file-diff delta measurement, and outcome back-propagation—each provide noisy, partial votes on window labels. Majority vote aggregation yields training labels with confidence scores that correlate strongly with human judgments (94% accuracy at confidence level 3, 83% at level 2).

As a downstream application of this framework, we train a lightweight GRU probe with attention over step embeddings. On the high-confidence subset (conf$\geq$2; 15,401 windows), the probe achieves 0.91 macro-F1, outperforming the strongest heuristic baseline Repeat-3 by 6.2 percentage points (pp). When trained on the full dataset including noisy conf=1 labels (54,809 windows), performance degrades to 0.82 F1 ($-$8.5pp), confirming that label noise is the primary bottleneck—not model capacity. Confidence-weighted training partially recovers this gap (+2pp), validating the framework's confidence signal.

On a 400-window human-annotated gold set (Fleiss κ=0.62), the probe reaches 0.77 F1—above the LF oracle (0.75) and Repeat-3 (0.72), but below an LLM-as-judge baseline (Claude few-shot: 0.82 F1). The probe's advantage lies in deployment efficiency: \$0.0005/window vs. \$0.018/window (35$\times$ cheaper) and 3ms vs. 2,800ms latency (930$\times$ faster), enabling real-time monitoring of large-scale agent runs.

Cross-agent transfer from SWE-agent to OpenHands yields 0.73 F1 zero-shot ($-$17.5pp), recoverable to 0.82 with just 200 labeled windows of few-shot fine-tuning. This demonstrates that the learned progress signal captures generalizable patterns rather than agent-specific behavioral artifacts.

Our contributions are:

1. A **weak-supervision framework** for agent trajectory annotation that transfers the data programming paradigm to temporal, non-i.i.d. trajectory signals—enabling scalable label generation without human annotation.

2. A systematic empirical study showing that a lightweight GRU probe trained on weakly-supervised labels achieves competitive progress estimation, capturing semantic stagnation patterns that rule-based heuristics miss.

3. Evidence for **cross-agent transferability** of learned progress signals, with efficient few-shot domain adaptation.

2 Related Work

Agent monitoring and error detection. Several lines of work address failure detection in LLM agents. Cycle detection methods identify structural repetition in action sequences but miss semantically diverse stagnation. Step-level value estimators such as AgentPRM and SWE-TRACE score individual actions; aggregating these scores over a window via uniform pooling discards the temporal signature of stagnation onset. Self-reflection approaches prompt the agent to evaluate its own progress, but a stagnating agent may lack the perspective to diagnose its failure mode.

Weak supervision and data programming. The Snorkel paradigm demonstrates that noisy, programmatic labeling functions can replace manual annotation when combined with a label model that estimates LF accuracies and correlations. Prior applications span text classification, relation extraction, and medical imaging. Our work extends data programming to agent trajectories, where LFs produce *temporal* signals over sliding windows rather than i.i.d. labels over independent instances.

Progress estimation. Goal-conditioned reinforcement learning uses value functions as implicit progress estimators, but requires reward specification. Our setting is reward-free: we estimate progress from trajectory structure alone, without access to task goals at inference time.

3 Method

3.1 Problem Formulation

Given an agent trajectory $\tau = (a_1, a_2, \ldots, a_T)$ of T actions, we extract sliding windows $w_i = (a_i, a_{i+1}, \ldots, a_{i+k-1})$ of k consecutive steps. Each window is classified as PROGRESS or STAGNATION. We use k=5 (Section 5.9).

3.2 Weak-Supervision Framework

We define three labeling functions, each capturing a different facet of stagnation:

LF_1: Command repetition. Detects windows where the agent issues structurally similar commands across steps. This is a *leading indicator*—command repetition typically precedes other stagnation signals by 1–2 steps (median lead: 1.5 steps). It fires first in 58% of stagnation windows.

LF_2: File-diff delta. Measures the magnitude of file changes within the window using git diff statistics. A near-zero delta indicates the agent is not modifying the codebase. This signal is approximately synchronous with stagnation onset (median lead: 0.5 steps).

LF_3: Outcome back-propagation. Propagates the trajectory-level outcome (success/failure) backward to assign soft labels to late-trajectory windows. This is a *lagging indicator* (median lag: 2.0 steps) that provides the only outcome-level signal in the framework.

Each LF abstains or votes PROGRESS/STAGNATION for each window. The final label is determined by majority vote. We define a confidence score $c \in \{1, 2, 3\}$ as the number of agreeing LFs. Windows with $c \geq 2$ (at least two LFs agree) constitute the high-confidence training set.

Label quality. On a 400-window human gold set (Section 5.2), label accuracy increases with confidence: 62% at c=1, 83% at c=2, and 94% at c=3. The c=1 subset—comprising 72% of all windows (39,408/54,809)—introduces substantial noise, predominantly in the stagnation class where a single conservative LF may mislabel exploratory behavior as stagnation.

3.3 Probe Architecture

We encode each action a_i using a frozen language model encoder, producing step embeddings $\mathbf{e}_i \in \mathbb{R}^{1024}$. A window $w = (a_1, \ldots, a_k)$ is represented as a sequence of embeddings $(\mathbf{e}_1, \ldots, \mathbf{e}_k)$.

A single-layer GRU processes the sequence:

$$\mathbf{h}_t = \text{GRU}(\mathbf{e}_t, \mathbf{h}_{t-1}), \quad t = 1, \ldots, k \tag{1}$$

Attention weights $\alpha_t = \text{softmax}(\mathbf{v}^\top \tanh(\mathbf{W}\mathbf{h}_t))$ produce the window representation $\mathbf{r} = \sum_t \alpha_t \mathbf{h}_t$. A linear classifier maps $\mathbf{r}$ to the binary prediction. The model has 542K parameters and runs in <3ms per window.

The architecture is a deliberate technical choice, not a contribution. Our experiments show that architecture selection matters less than label quality (Section 5.9).

4 Experimental Setup

Data source. We use SWE-bench Lite agent trajectories generated by SWE-agent. After sliding-window extraction (k=5, stride 1), we obtain 54,809 windows total, of which 15,401 have confidence $c \geq 2$.

Setting	Macro-F1	Stag-F1	Prog-F1
conf$\geq$2 only	0.906	0.859	0.952
Full (conf$\geq$1)	0.820	0.760	0.880
Conf-weighted	0.840	0.790	0.890
Repeat-3 (conf$\geq$2)	0.843	0.750	0.936
Repeat-3 (full)	0.785	—	—

Table 1: Probe performance under different training regimes. Full-data training degrades by 8.5pp due to noisy conf=1 labels. Confidence-weighted loss recovers 2pp.

Baselines. (1) **Repeat-3**: flags a window as stagnation if $\geq$3 of 5 actions share the same command template. (2) **Cosine similarity**: average pairwise cosine similarity of step embeddings; high similarity $\Rightarrow$ stagnation. (3) **MeanPool+LR**: mean-pooled step embeddings classified by logistic regression. (4) **LF Oracle**: the majority-vote label at test time (a ceiling for the labeling functions themselves). (5) **LLM-as-Judge**: Claude-3.5-Sonnet and GPT-4o in zero-shot and few-shot settings. (6) **Structural features**: 15 hand-crafted numerical features (lines changed, unique command ratio, file count delta, etc.) with a gradient-boosted classifier. (7) **Random** and **Majority** baselines.

Human gold set. 400 windows annotated by 3 annotators (Fleiss κ=0.62, 78% agreement). Gold labels serve as the primary evaluation for breaking circular evaluation.

5 Results

5.1 Main Results on Auto-Labeled Data

Table 1 presents the probe trained under three regimes. On the high-confidence subset (conf$\geq$2; 15,401 windows), the probe achieves 0.906 macro-F1, outperforming Repeat-3 by 6.2pp. The gap is largest for stagnation detection (+10.9pp), where temporal modeling captures patterns beyond structural repetition.

Training on the full dataset including noisy conf=1 labels drops performance to 0.82 ($-$8.5pp). The gap vs. Repeat-3 narrows from 6.2pp to 3.5pp. Noise is asymmetric: stagnation F1 drops 10pp (0.86$\rightarrow$0.76) while progress F1 drops 7pp (0.95$\rightarrow$0.88), because conf=1 windows contain many false stagnation labels from a single conservative LF.

Confidence-weighted training (loss weights: conf3=1.0, conf2=0.7, conf1=0.3) partially recovers to 0.84 (+2pp over unweighted full-data training), demonstrating that the framework's confidence scores carry actionable signal for downstream model training.

5.2 Evaluation on Human Gold Labels

Table 2 breaks the circular evaluation concern by comparing all methods on human gold labels. The probe (0.77 F1) surpasses the LF oracle (+2pp) and Repeat-3 (+5pp), confirming that learning adds value beyond the labeling functions. The LLM judge (Claude few-shot, 0.82 F1) outperforms the probe by 5pp, reflecting the advantage of direct semantic understanding unmediated by LF bias.

The probe's value proposition is deployment efficiency. Monitoring 54K windows costs $27 with the probe vs. $972 with the LLM judge (35$\times$). Inference latency is 3ms vs. 2,800ms (930$\times$), enabling real-time, per-step monitoring that is infeasible with API-based LLM judges.

Method	Gold F1	Stag-F1	Prog-F1	$/win
LLM Judge (FS)	0.820	0.780	0.860	0.018
Probe (conf$\geq$2)	0.770	0.710	0.830	0.0005
LF Oracle	0.750	0.700	0.800	$\sim$0
MeanPool+LR	0.740	0.680	0.800	0.0005
Repeat-3	0.720	0.650	0.790	$\sim$0
Cosine-sim	0.380	0.300	0.460	$\sim$0

Table 2: All methods evaluated on the 400-window human gold set (κ=0.62). The probe is the best non-LLM method; the LLM judge leads at 35$\times$ higher cost.

Method	Auto F1	Gold F1	$/win	ms/win
Probe (conf$\geq$2)	0.906	0.77	0.0005	3
Probe (full)	0.820	0.77	0.0005	3
Claude ZS	0.810	0.79	0.018	2800
Claude FS	0.840	0.82	0.018	2800
GPT-4o ZS	0.800	0.78	0.015	3200
GPT-4o FS	0.830	0.80	0.015	3200
LF Oracle	1.000	0.75	$\sim$0	$\sim$0

Table 3: Cost–quality tradeoff between the probe and LLM judges. The probe operates on a fundamentally different cost–accuracy Pareto frontier.

Label quality by confidence. Auto-label accuracy on the gold set is 62% (c=1), 83% (c=2), and 94% (c=3), validating confidence as a quality proxy and explaining the 8.5pp full-data performance drop.

5.3 LLM-as-Judge Comparison

Table 3 situates the probe within the cost–quality landscape. The LLM judge's 5pp gold-F1 advantage comes at 35$\times$ cost and 930$\times$ latency. On auto-labeled data, the probe (conf$\geq$2) substantially outperforms all LLM judges (0.906 vs. 0.84), but this advantage partly reflects alignment with the LF-generated labels. The probe and the LLM judge serve different deployment regimes: the probe enables always-on monitoring at scale; the LLM judge is appropriate for high-stakes, low-volume evaluation.

5.4 LF Oracle Baseline

The LF oracle achieves perfect F1 on auto-labeled test data by construction but only 0.75 on human gold labels. The probe exceeds the oracle on gold (+2pp), particularly on conf=1 windows (+8pp: 0.68 vs. 0.60), indicating that the probe generalizes beyond its label source in ambiguous cases. On conf$\geq$2 gold windows, the oracle matches the probe (0.84 vs. 0.83, $-$1pp), suggesting that when LFs agree and are correct, the probe adds no incremental value.

5.5 Non-NLP Feature Baseline

Table 4 compares purely structural features (lines changed, unique command ratio, file count delta, etc.) against NL embeddings. The 18.5pp gap—especially the 23pp gap on stagnation detection—demonstrates that progress estimation is not solvable by structural metrics alone. Semantic understanding of action content is essential. Adding structural features to embeddings yields only +0.5pp, indicating that the embedding space already captures the relevant structural signals.

Feature Set	Macro-F1	Stag-F1
Structural only (15 features)	0.720	0.630
NL Embedding (GRU)	0.906	0.859
Structural + Embedding	0.910	0.870

Table 4: NLP vs. non-NLP features. The 18.5pp gap demonstrates that natural language understanding is necessary for progress estimation.

Setting	Macro-F1	Stag-F1	Prog-F1
SWE-agent (conf$\geq$2)	0.906	0.859	0.952
OpenHands zero-shot	0.730	0.650	0.810
OpenHands 50-shot	0.780	0.720	0.840
OpenHands 200-shot	0.820	0.770	0.870
OpenHands 500-shot	0.850	0.810	0.890
Random	0.500	—	—
Repeat-3 (OpenHands)	0.710	—	—

Table 5: Cross-agent transfer from SWE-agent to OpenHands. Zero-shot transfer is viable (+23pp vs. random) but domain adaptation improves performance substantially.

5.6 Cross-Agent Transfer

Table 5 evaluates the SWE-agent-trained probe on 2,140 OpenHands trajectories (18,720 windows). Zero-shot F1 is 0.73—a 17.5pp drop from SWE-agent performance, but 23pp above random and 2pp above Repeat-3 on OpenHands. The drop stems from action-space differences (different command formats and file operation APIs) and incomplete LF adaptation (command repetition LF recall drops 15% due to format mismatch).

Few-shot fine-tuning efficiently recovers performance: 50 labeled windows reach 0.78, 200 windows reach 0.82, and 500 windows approach source-domain performance at 0.85. This suggests the learned representations provide a strong initialization that transfers the general structure of progress signals, requiring only modest target-domain calibration. Stagnation F1 transfers less well than progress F1 (0.65 vs. 0.81 zero-shot), consistent with the intuition that agents stagnate in implementation-specific ways while progress patterns are more universal.

5.7 Error Analysis

Table 6 categorizes probe errors on the conf$\geq$2 test set. The dominant false-positive mode is misclassifying divergent exploration as stagnation (42%)—the agent searches different files without linear progress, which the probe cannot distinguish from genuine stagnation without goal awareness. The dominant false-negative mode is semantic stagnation without repetition (58%)—the agent issues diverse commands that make no progress; 71% of these are also missed by Repeat-3.

Complementarity with Repeat-3. The probe correctly classifies 187 windows that Repeat-3 misses (paraphrased repetition, gradual onset stagnation), while Repeat-3 catches only 34 windows the probe misses. This asymmetry confirms that the learned temporal model captures signals beyond structural repetition.

Category	Count	%	Pattern
False Positives (98 total)			
Exploration → stag.	41	42%	grep/find diverse files
LF conflict	30	31%	conf=2, LFs disagree
Other	27	27%	edge cases
False Negatives (123 total)			
Semantic stagnation	72	58%	diverse cmds, zero progress
Late onset	31	25%	stag. in last 1–2 steps
Other	20	16%	rare patterns

Table 6: Error taxonomy on the conf$\geq$2 test set (221 errors, 7.2% error rate).

Removed LF	F1	Drop	conf$\geq$2	Flips%
None	0.906	—	15,401	—
Command rep.	0.870	$-$3.5pp	9,800	12.3%
File diff	0.880	$-$2.5pp	10,200	8.7%
Outcome BP	0.890	$-$1.5pp	12,100	5.1%

Table 7: LF hold-one-out ablation. Each LF contributes 1.5–3.5pp; none is redundant.

5.8 Labeling Function Analysis

Table 7 validates the framework design: each LF contributes uniquely (1.5–3.5pp drop upon removal), and no single LF is redundant. Command repetition is the backbone LF (largest drop, 12.3% label flips upon removal), but removing it still leaves the probe at 0.87—well above Repeat-3 (0.84)—confirming that the probe learns more than repetition detection.

The three LFs exhibit distinct temporal roles: command repetition is a leading indicator (median 1.5-step lead), file-diff delta is approximately synchronous, and outcome back-propagation is lagging (median 2-step lag). LF agreement is low at stagnation onset (0.35) but converges during deep stagnation (0.72), explaining why conf=1 windows concentrate at onset boundaries where only the leading LF has fired.

5.9 Ablation Studies

Temporal modeling. Table 8 shows that architecture choice matters less than label quality. The GRU+attention advantage over MeanPool+LR (+4.5pp) confirms temporal modeling has value—particularly for stagnation detection (+5pp)—but the narrow range across architectures (0.87–0.91) places the emphasis on the framework rather than the model.

Attention analysis reveals that stagnation windows concentrate attention on steps 3–4 (late window), consistent with stagnation onset patterns, while progress windows distribute attention more uniformly (entropy 1.8 vs. 1.2). This provides interpretive evidence for the temporal modeling claim.

Window size. k=5 is optimal (0.906 F1). k=3 (0.86) and k=7 (0.89) degrade modestly (<5pp), indicating robustness to window size within $\pm$2 steps.

Data scaling. Table 9 shows approximately log-linear scaling. The probe surpasses Repeat-3 at $\sim$25% of the data ($\sim$4K windows), establishing the minimum viable dataset size. Diminishing returns set in at 50% ($\sim$7.7K windows), suggesting the conf$\geq$2 subset is near saturation—further gains require better LFs or label models rather than more data.

Architecture	F1	Stag-F1	Params	ms
GRU+Attention	0.906	0.859	542K	2.8
GRU+MeanPool	0.890	0.840	530K	2.5
Transformer	0.900	0.850	1.2M	3.5
MLP-Concat	0.870	0.820	2.6M	1.2
MeanPool+LR	0.860	0.810	—	—

Table 8: Architecture ablation on conf$\geq$2 data. The 3.5pp performance range across architectures is smaller than the 8.5pp impact of label noise.

Data %	Windows	F1	vs. R3
10%	1,540	0.800	$-$4.3pp
25%	3,850	0.850	+0.7pp
50%	7,700	0.880	+3.7pp
100%	15,401	0.906	+6.2pp

Table 9: Data scaling curve (conf$\geq$2). The probe surpasses Repeat-3 at $\sim$25% ($\sim$4K windows), demonstrating the framework's data efficiency.

Supervised contrastive loss. Adding a SupCon auxiliary loss (λ=0.3) improves conf$\geq$2 F1 by 1.0pp (to 0.915) and full-data F1 by 1.5pp (to 0.835). The embedding silhouette score increases from 0.42 to 0.58. The larger gain on noisy data suggests contrastive learning provides implicit robustness to label noise.

6 Discussion

Framework as primary contribution. The experiments consistently show that label quality—determined by the weak-supervision framework—is the dominant factor in probe performance. The 8.5pp gap between conf$\geq$2 and full-data training far exceeds the impact of architecture choice (3.5pp range), window size (4.5pp range), or auxiliary losses (1–1.5pp). This supports our framing of the weak-supervision framework as the primary contribution, with the probe as one efficient downstream application.

When to use the probe vs. an LLM judge. The LLM judge achieves higher gold-F1 (0.82 vs. 0.77) but at 35$\times$ cost and 930$\times$ latency. For always-on monitoring of production agent runs (thousands of trajectories), the probe is the practical choice. For post-hoc analysis of critical failures, the LLM judge provides higher accuracy. The framework supports both: it generates training data for probes and evaluation data for LLM judges.

Early termination. A simulation on 320 held-out trajectories shows that probe-triggered early termination at threshold 0.7 saves 22% of tokens with only 3.1% success rate degradation. However, 8.7% of terminated trajectories would have eventually recovered—the *intervention paradox* inherent to any real-time monitor. Conservative thresholds (0.9) reduce false terminations to 1.2% success drop at 12% savings.

Interpretability. t-SNE visualization of GRU hidden states shows partial but meaningful cluster separation (silhouette: 0.42) with 18% boundary overlap concentrated in ambiguous exploration-vs-stagnation windows. Attention weights align with LF firing positions 68% of the time, with the remaining 32% representing signals the probe learns beyond the LFs.

7 Limitations

Scope. Both agents (SWE-agent, OpenHands) operate on software engineering tasks. Transfer to non-SE domains (web navigation, scientific research) is untested and likely requires new LFs.

Goal awareness. The probe and LFs operate without explicit task goals. The largest error category—goal-unaware semantic stagnation (58% of FN)—requires task-level understanding that our framework does not provide. Goal-conditioned extensions are a clear direction.

LF format dependency. The command repetition LF depends on agent-specific command formats, causing 15% recall degradation on OpenHands. More format-agnostic LFs would improve framework portability.

Gold set size. The 400-window gold set provides moderate statistical power. The 2pp probe–oracle gap on gold (N=400) has wide bootstrap confidence intervals; a larger gold set would tighten claims.

Label noise ceiling. Conf=1 labels have 62% accuracy—near random for the minority class. More sophisticated label models (e.g., learning LF accuracies) could improve upon majority vote, but we leave this to future work.

8 Conclusion

We presented a weak-supervision framework for annotating LLM agent trajectories with window-level progress labels. Three labeling functions with complementary temporal dynamics generate training data at scale, with confidence scores that reliably predict label quality (62%–94% accuracy across confidence levels). A lightweight GRU probe trained on this data achieves competitive progress estimation—outperforming heuristic baselines while operating at 35$\times$ lower cost than LLM judges—and transfers across agent implementations with efficient few-shot adaptation.

The framework's primary value is making progress estimation research feasible without expensive manual annotation. The probe is one instantiation; the labeled data can equally train other architectures, calibrate LLM judges, or serve as features for higher-level agent orchestrators. We release the framework, labeled dataset, and probe weights to support further research on agent monitoring.

A Window Size Ablation Details

k	Windows	Macro-F1	Stag-F1	Prog-F1
3	22,130	0.860	0.810	0.910
5	15,401	0.906	0.859	0.952
7	11,280	0.890	0.840	0.940

Table 10: Full window size ablation.

B SupCon Ablation Details

Setting	**w/o SupCon**	**w/ SupCon**	**Gain**
conf$\geq$2 F1	0.906	0.915	+1.0pp
Full F1	0.820	0.835	+1.5pp
Silhouette	0.42	0.58	+0.16

Table 11: Supervised contrastive loss ablation.

C Early Termination Simulation

Threshold	**Token Savings**	**Success Drop**	**False Term.**
0.5	35%	-7.8%	25
0.7	22%	-3.1%	10
0.9	12%	-1.2%	4

Table 12: Early termination tradeoff on 320 held-out trajectories.

D LF Temporal Dynamics

LF	Med. Lead/Lag	First-Fire %	Type
Command rep.	+1.5 steps	58%	Leading
File diff	+0.5 steps	28%	Synchronous
Outcome BP	-2.0 steps	14%	Lagging

Table 13: Temporal characteristics of labeling functions relative to stagnation onset.